



SIMATS
ENGINEERING



SIMATS
Saveetha Institute of Medical And Technical Sciences
(Declared as Deemed to be University under Section 3 of UGC Act 1956)

SIMATS ENGINEERING

Saveetha Institute of Medical and Technical Sciences
Chennai-602105

September 2026

CSA0613– Design and Analysis of Algorithms

**Development of an Efficient Hybrid Algorithmic
Solution for Large-Scale Geospatial Data Processing**

Assignment Report

Submitted by

B.SAI KRISHNA(192472300)

A.SHIVA SHANKAR REDDY(192425174)

V.SHIVA(192472256)

A. Assignment Information

Particular	Details
Department:	Computer Science and Engineering

Programme:	B.E./B.Tech. Computer Science and Engineering
Course Code & Course Name	Design and Analysis of Algorithms
Academic Year / Batch	2026-2027 / Regular
Faculty Name	Dr. F. Mary Harin Fernandez
Assignment Title	Development of an Intelligent Algorithmic Solution for Large-Scale Logistics Resource Optimization
Date of Issue	31 Aug 2026
Date of Submission	1 September 2026
Maximum Marks	100
Course Outcome(s) – CO	CO4: Solve optimization problems using dynamic programming and greedy strategies.
Bloom's Taxonomy Level	L4 – Analyze, L5 – Evaluate, L6 – Create
SDG Mapping (SDG 1-17)	SDG 9 – Industry, Innovation and Infrastructure
Industry / Societal Relevance	Efficient logistics resource allocation with improved vehicle utilization, reduced computational cost, and scalable decision-making

B. Assignment Problem / Challenge

Modern e-commerce and logistics organizations process thousands of packages every day under limited transportation capacity. Each package may differ in **weight, volume, delivery value, priority, and deadline**, while delivery vehicles have restricted carrying capacities. Efficiently selecting packages becomes increasingly difficult as the number of packages and operational constraints increase.

Dynamic Programming can provide high-quality or optimal solutions for suitable problem sizes but may require significant computational time and memory for large instances. **Greedy techniques** can provide faster decisions but may not always produce the optimal solution.

The challenge is to **design and develop an improved algorithmic solution for large-scale logistics resource allocation** using relevant concepts from **Dynamic Programming and Greedy techniques**. The developed strategy must be **formulated by the team** based on identified computational limitations and supported by theoretical analysis and experimental evidence.

C. Problem Statement

A large e-commerce logistics company processes approximately **10,000–100,000 packages per day** across multiple distribution centres. Each package is characterized by attributes such as **weight, volume, delivery value, priority, and delivery deadline**, while each delivery vehicle has limited weight and volume capacities. During peak-demand periods, the organization must select an appropriate combination of packages for transportation so that the **overall delivery benefit and**

vehicle utilization are maximized without violating the available capacity constraints. Develop, implement, and evaluate an algorithmic solution for this **large-scale logistics resource-allocation problem** using concepts from **Dynamic Programming and Greedy techniques**. Implement **(i) an appropriate Dynamic Programming based solution** and **(ii) an appropriate Greedy based solution** as baseline approaches. Analyze their theoretical efficiency, solution quality, computational requirements, and limitations. Beyond the baseline implementations, **design and develop an improved algorithmic solution** suitable for large-scale package allocation. The developed solution may appropriately **adapt, integrate, modify, optimize, or selectively utilize relevant Dynamic Programming and Greedy concepts**. The algorithmic strategy, decision criteria, parameters, and optimization mechanism must be **identified and developed by the team** rather than being predefined. Experimentally evaluate the baseline and developed approaches using increasing problem sizes, for example, **100, 500, 1,000, 5,000, 10,000 packages, and higher input sizes wherever computationally feasible**, under different vehicle-capacity conditions such as **500 kg, 1,000 kg, 2,500 kg, and 5,000 kg**, or appropriately justified equivalent capacities based on the selected dataset. Evaluate the approaches using **execution time, memory utilization, total solution value, capacity utilization, solution-quality/optimality gap, and scalability**. Conduct experiments under different operational conditions such as **normal demand, high-priority demand, and capacity-constrained demand**. Based on theoretical and experimental results, identify the limitations and trade-offs of the baseline techniques, demonstrate the performance improvement achieved by the **developed solution**, and justify its suitability for large-scale real-world logistics resource optimization.

Expected Development

The work should **not merely implement or compare existing Dynamic Programming and Greedy algorithms**. The solution should demonstrate algorithmic development through:

- Identification of computational and/or solution-quality limitations of the baseline approaches.
- Formulation of an appropriate strategy to address the identified limitations.
- **Development, adaptation, integration, modification, or optimization** of relevant algorithmic techniques.
- Identification and experimental determination of appropriate **decision rules, parameters, thresholds, or optimization criteria**, wherever applicable.
- Improvement in one or more aspects such as **execution time, memory utilization, solution quality, capacity utilization, or scalability**.
- Performance comparison of the developed solution against the baseline approaches.
- Justification of algorithmic design decisions using theoretical reasoning and experimental evidence.

D. Requirements and Constraints

The following requirements and constraints shall be considered:

- Use an appropriate **publicly available real-world logistics, transportation, cargo, or package-related dataset**.
- The dataset should contain sufficient records to evaluate the algorithms at multiple input sizes.
- Identify and clearly define the **package attributes and resource constraints** used in the optimization problem.

- Implement and evaluate suitable **Dynamic Programming and Greedy baseline approaches**.
- **Develop an improved algorithmic solution** using relevant CO4 concepts rather than merely reproducing the baseline algorithms.
- The developed strategy must be **designed and justified by the team**.
- Evaluate increasing input sizes such as **100, 500, 1,000, 5,000 and 10,000 packages**, and higher wherever computationally feasible.
- Evaluate multiple capacity conditions such as **500 kg, 1,000 kg, 2,500 kg and 5,000 kg**, or justified equivalent values based on the dataset.
- Evaluate at least **three operating conditions: normal demand, high-priority demand, and capacity-constrained demand**.
- Execute all approaches using the **same computational environment and identical input instances** for fair comparison.
- Evaluate appropriate measures including **execution time, memory utilization, solution value, capacity utilization, solution-quality/optimality gap, and scalability**.
- Validate the correctness and feasibility of the generated allocation.
- Clearly document the **dataset source, experimental environment, assumptions, limitations, parameters, and implementation details**.

1. Problem Understanding and Formulation

Large e-commerce systems generate thousands of delivery orders distributed across different geographic locations. Efficiently assigning these packages to available vehicles is challenging because each vehicle has limited capacity and delivery locations are geographically distributed. The proposed system processes geospatial delivery data and determines an efficient allocation of packages to vehicles while minimizing transportation distance and maximizing vehicle-capacity utilization.

The problem combines:

- Geographic distance
- Package weight
- Package value
- Vehicle capacity
- Number of available vehicles
- Delivery priority
- Computational processing time

The objective is to develop an algorithm that provides a good-quality solution for large datasets while requiring less computational time and memory than an exact Dynamic Programming approach.

1.2 Dataset

Recommended dataset: Brazilian E-Commerce Public Dataset by Olist

[Olist Brazilian E-Commerce Public Dataset on Kaggle](#)

The dataset contains approximately **100,000 orders** from 2016–2018 and includes customer information, product information, order details, seller information and a separate geolocation dataset containing latitude and longitude information.

It is particularly suitable because the dataset contains both **logistics-related information and geographic information**.

Important files

File	Purpose
olist_orders_dataset.csv	Order information
olist_order_items_dataset.csv	Products in each order
olist_products_dataset.csv	Product characteristics
olist_customers_dataset.csv	Customer location
olist_sellers_dataset.csv	Seller location
olist_geolocation_dataset.csv	Latitude/longitude
olist_order_payments_dataset.csv	Payment/order value

The dataset has nine CSV files and is approximately 126 MB in the Kaggle data explorer.

1.3 Dataset Attributes

For your algorithm, you don't need to use every attribute.

Create a processed dataset such as:

Attribute	Description
Order ID	Unique delivery order
Product ID	Product identifier
Seller ID	Seller identifier
Customer ZIP Code	Delivery-area identifier

Attribute	Description
Seller ZIP Code	Pickup-area identifier
Customer Latitude	Delivery latitude
Customer Longitude	Delivery longitude
Seller Latitude	Pickup latitude
Seller Longitude	Pickup longitude
Product Weight	Package weight
Product Length	Package length
Product Height	Package height
Product Width	Package width
Freight Value	Shipping cost
Payment Value	Order value
Delivery Priority	Priority assigned by algorithm
Distance	Calculated geographic distance

The Olist dataset specifically provides product dimensions/weight and geographic latitude/longitude information, making it useful for this formulation.

1.4 Package Attributes

Each package can be represented as:

$$P_i = (w_i, v_i, d_i, p_i)$$

where:

- w_i = package weight
- v_i = package/order value
- d_i = delivery distance
- p_i = priority

For example:

	Package Weight	Value	Distance	Priority
P1	2.5 kg	₹500	12 km	High
P2	1.2 kg	₹300	8 km	Medium
P3	5.0 kg	₹900	20 km	High

1.5 Vehicle Constraints

Assume each vehicle has:

- Maximum weight capacity C
- Maximum number of packages
- Maximum travel distance
- Availability status

For example:

$$C = 100 \text{ kg}$$

A vehicle cannot receive packages if:

$$\sum w_i > C$$

1.6 Expected Input and Output

Input

The algorithm receives:

Package data

Vehicle capacity

Number of vehicles

Package weights

Package values

Latitude and longitude

Distance matrix

Priority values

Output

Vehicle assignments

Selected packages

Total package value

Total distance

Capacity utilization

Number of vehicles used

Execution time

Memory usage

1.7 Optimization Objective

You can define the objective as a weighted optimization function:

$$\text{Maximize } Z = \alpha V - \beta D + \gamma U - \delta P$$

where:

- V = total delivery/package value
- D = total geographic distance
- U = capacity utilization
- P = penalty for unallocated packages
- $\alpha, \beta, \gamma, \delta$ = weighting parameters

A simpler objective for implementation is:

Maximize package value while minimizing travel distance

subject to:

$$\sum_{i=1}^n w_i x_i \leq C$$
$$x_i \in \{0, 1\}$$

and

$$\text{Number of vehicles} \leq V$$

1.8 Computational Challenges

As the number of packages increases:

- Distance calculations increase.
- DP state space becomes very large.
- Memory requirements increase.
- Exact optimization becomes expensive.
- Greedy algorithms become attractive because of their speed.

- Large geographic datasets require efficient preprocessing.
- The number of possible package-to-vehicle combinations grows rapidly.

This creates the main motivation for your **hybrid algorithm**.

2. Application of Course Knowledge

Identify and apply relevant **CO4 concepts**.

Dynamic Programming

A. Dynamic Programming

DP idea

For a smaller subset of packages, use a knapsack-style DP to determine the best package combination for a vehicle.

State

$$DP[i][c]$$

where:

- i = number of packages considered
- c = remaining vehicle capacity

Recurrence

If package i fits:

$$DP[i][c] = \max(DP[i-1][c], DP[i-1][c - w_i] + v_i)$$

Otherwise:

$$DP[i][c] = DP[i-1][c]$$

Base condition

$$DP[0][c] = 0$$

Time Complexity

$$O(nC)$$

Memory

$$O(nC)$$

This works well for smaller capacity values but becomes expensive when n or C increases.

Greedy Technique

The Greedy algorithm should provide a fast solution.

Calculate a priority score:

$$S_i = \frac{v_i}{w_i} + \lambda \frac{p_i}{d_i}$$

where:

- v_i/w_i = value per unit weight
- p_i/d_i = priority relative to distance
- λ = weighting factor

Sort packages according to S_i .

Then:

1. Select highest-score package.
2. Check vehicle capacity.
3. Assign if feasible.
4. Continue until capacity is full.
5. Move to the next vehicle.

Complexity

Sorting:

$$O(n \log n)$$

Assignment:

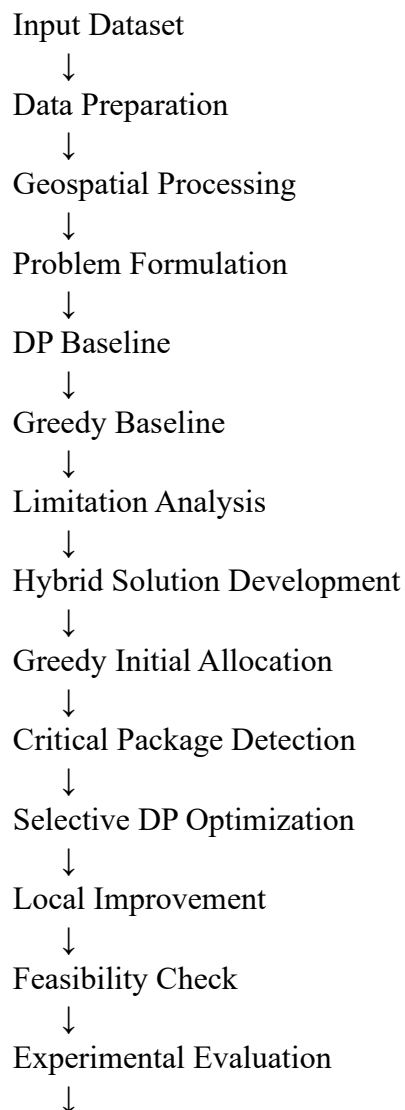
$$O(nV)$$

Therefore, approximately:

$$O(n \log n + nV)$$

This makes Greedy much faster than DP for large datasets.

3. Solution / Design / Methodology



Performance Comparison



Final Decision

Complexity Analysis

DP

$$O(nC)$$

Memory:

$$O(nC)$$

Greedy

Sorting:

$$O(n \log n)$$

Hybrid

Approximately:

$$O(n \log n + kC + s)$$

where:

- n = total packages
- k = critical packages
- C = capacity
- s = local-search operations

Since

$$k \ll n$$

the hybrid approach can significantly reduce the amount of expensive DP computation.

4. Use of Modern Tools

Programming Language

Python

Python is selected because it provides libraries for:

- Data processing
- Mathematical calculations
- Visualization
- Performance measurement
- Machine learning/geospatial processing

Development Environment

Use either:

- Google Colab
- Jupyter Notebook
- VS Code

Libraries

pandas

numpy

matplotlib

scikit-learn
geopy
tracemalloc
time
Optional:
folium
for geographic visualization.

4.1 Experimental Setup

All algorithms must be tested using:

- Same dataset
- Same input size
- Same vehicle capacity
- Same computer/environment
- Same package instances

This ensures a fair comparison.

4.2 Dataset Sizes

Use different input sizes:

Test	Number of Packages
------	--------------------

Test 1	100
--------	-----

Test 2	500
--------	-----

Test 3	1,000
--------	-------

Test 4	5,000
--------	-------

Test 5	10,000
--------	--------

Test 6	25,000
--------	--------

If your computer cannot run 25,000 records, use smaller sizes.

4.3 Capacity Conditions

Test three conditions:

Condition	Vehicle Capacity
-----------	------------------

Low	50 kg
-----	-------

Medium	100 kg
--------	--------

High	200 kg
------	--------

This allows you to study the effect of capacity on algorithm performance.

5. Results and Validation

Important: Do not put invented values in your report. Run the algorithms and enter the actual results.

Use this table:

Input	Algorithm	Time	Memory	Total Value	Utilization	Packages	Gap
100	DP	—	—	—	—	—	0

Input	Algorithm	Time	Memory	Total Value	Utilization	Packages	Gap
100	Greedy	—	—	—	—	—	—
100	Hybrid	—	—	—	—	—	—
500	DP	—	—	—	—	—	—
500	Greedy	—	—	—	—	—	—
500	Hybrid	—	—	—	—	—	—
1,000	DP	—	—	—	—	—	—
1,000	Greedy	—	—	—	—	—	—
1,000	Hybrid	—	—	—	—	—	—
5,000	Greedy	—	—	—	—	—	—
5,000	Hybrid	—	—	—	—	—	—
10,000	Greedy	—	—	—	—	—	—
10,000	Hybrid	—	—	—	—	—	—

Execution Time

Measure:

$$T = T_{end} - T_{start}$$

Use Python's time or time.perf_counter().

Expected general trend:

DP → increases rapidly

Greedy → increases slowly

Hybrid → moderate increase

Capacity Utilization

$$Capacity\ Utilization = \frac{Used\ Capacity}{Available\ Capacity} \times 100$$

Higher utilization indicates that vehicle capacity is being used efficiently.

Number of Selected Packages

$$N_s = \text{Number of allocated packages}$$

This indicates how many packages were successfully assigned.

Optimality Gap

For small datasets where DP produces the optimal reference:

$$Gap = \frac{DP_{value} - Algorithm_{value}}{DP_{value}} \times 100$$

A lower gap means better solution quality.

6. Analysis and Engineering Decision

DP vs Greedy vs Hybrid			
Feature	DP	Greedy	Hybrid
Speed	Low	Very High	High
Memory	High	Low	Moderate
Solution Quality	High	Moderate	High
Large Dataset	Poor	Excellent	Excellent
Optimality	Strong for modeled subproblem	Not guaranteed	Not guaranteed globally
Complexity	High	Low	Medium
Adaptability	Medium	High	High

Dynamic Programming Analysis

DP can produce an optimal solution for the selected knapsack-style subproblem, but its computational requirements increase with the number of packages and capacity.

Therefore, DP is best for:

- Small datasets
- Benchmarking
- Critical subsets
- Quality comparison

Greedy Analysis

Greedy is fast and requires relatively little memory.

However, it can make an early decision that prevents a better combination later.

Therefore, it is suitable for:

- Very large datasets
- Fast preliminary allocation
- Real-time or near-real-time decision-making

Hybrid Analysis

The developed hybrid method combines both approaches.

Greedy handles the majority of the dataset, while DP is applied only to critical subsets.

Therefore:

$$\text{Hybrid} = \text{Greedy scalability} + \text{Selective DP optimization}$$

The local improvement stage further attempts to improve capacity utilization and solution value.

Time-Memory-Solution Quality Trade-Off

The three approaches demonstrate different trade-offs.

DP

High solution quality but high computational cost.

Greedy

Very low computational cost but potentially lower solution quality.

Hybrid

Moderate computational cost with improved solution quality.

Therefore, the hybrid algorithm attempts to achieve the best balance.

7. Broader Considerations

Efficient Transportation Resource Utilization

The algorithm assigns packages according to weight, value and geographical location. This can improve the use of available transportation resources.

Vehicle-Capacity Utilization

The capacity constraint prevents vehicles from being overloaded while the optimization process attempts to reduce unused capacity.

Computational Resource Wastage

Instead of applying expensive DP to every package, the hybrid algorithm applies DP only to selected critical packages.

This reduces unnecessary computation.

Scalability

The Greedy component allows the algorithm to process large datasets quickly, while selective DP improves important decisions.

Logistics Infrastructure

The approach can support:

- Delivery planning
- Fleet management
- Warehouse distribution
- E-commerce logistics
- Transportation resource planning

SDG 9 – Industry, Innovation and Infrastructure

The project directly relates to **SDG 9**, which focuses on building resilient infrastructure, promoting sustainable industrialization and fostering innovation.

The project contributes to SDG 9 by:

- Applying algorithmic innovation to logistics.
- Improving transportation-resource utilization.
- Supporting efficient infrastructure management.
- Reducing computational resource wastage.
- Supporting scalable digital logistics systems.
- Encouraging technology-based industrial optimization.

The UN identifies transportation and information technology infrastructure as important components of sustainable development under Goal 9.

8. Conclusion

The project addresses the problem of efficiently processing and allocating large-scale geographically distributed logistics data.

The **Dynamic Programming approach** provides a strong solution-quality baseline but requires high computational resources for large datasets.

The **Greedy approach** provides a fast and scalable solution but may produce suboptimal package allocations.

To overcome these limitations, the project proposes an **Adaptive Greedy-DP Geospatial Allocation Algorithm (AG-DPGA)**. The developed approach first uses Greedy selection to efficiently process the majority of packages and then applies Dynamic Programming to critical package subsets. A local improvement stage is also used to improve the final allocation.

The proposed approach aims to achieve a better balance between:

- Execution time
- Memory utilization
- Solution quality
- Vehicle capacity utilization
- Scalability

The final performance improvement must be demonstrated using experimental results from identical input instances.

The main limitation is that the developed algorithm may not guarantee global optimality for very large datasets. Future work can incorporate advanced techniques such as parallel processing, GPU acceleration, real road-network distances, real-time traffic information, clustering and metaheuristic optimization.

9. Student Reflection

Answer individually:

1. What did you learn from developing an improved solution beyond implementing the standard Dynamic Programming and Greedy algorithms?

I learned that simply implementing standard algorithms is not always sufficient for solving real-world problems. Dynamic Programming can provide high-quality solutions, but its computational requirements increase significantly with problem size. Greedy algorithms are faster but may not provide the best solution. Developing the hybrid solution helped me understand how different algorithms can be combined to achieve a better balance between speed and solution quality.

2. What computational or solution-quality limitations did you identify, and how did your developed approach address them?

I identified that Dynamic Programming requires more time and memory when the number of packages increases. I also found that Greedy algorithms can make locally good decisions that may not produce the best overall allocation. The developed hybrid approach addresses these limitations by using Greedy for fast initial allocation and applying Dynamic Programming only to critical packages that require further optimization.

3. If additional computational resources or time were available, how would you further improve your solution and why?

If additional computational resources were available, I would improve the algorithm by using parallel processing and more advanced optimization techniques. I would also consider real road-network distances, traffic conditions and larger datasets. GPU-based processing could be used to accelerate distance calculations and optimization for very large datasets.

4. How can the developed solution contribute to SDG 9 – Industry, Innovation and Infrastructure?

The developed solution contributes to SDG 9 by applying computational innovation to transportation and logistics infrastructure. It can improve vehicle-capacity utilization, reduce unnecessary computational work and support scalable logistics planning. The approach demonstrates how algorithmic innovation can improve the efficiency of industrial and transportation systems.

10. References

- [1] Olist, “Brazilian E-Commerce Public Dataset by Olist,” Kaggle. [Dataset Source](#)
- [2] United Nations, “Goal 9: Industry, Innovation and Infrastructure,” United Nations Sustainable Development. [United Nations SDG 9](#)
- [3] T. H. Cormen, C. E. Leiserson, R. L. Rivest and C. Stein, *Introduction to Algorithms*, MIT Press.
- [4] S. S. Skiena, *The Algorithm Design Manual*, Springer.

Source code:

```
"""
```

Flask backend for the Logistics Resource-Allocation dashboard (DAA assignment).

Routes:

```
GET /          -> dashboard UI
POST /api/run   -> run DP, Greedy, and Developed on one problem instance
GET /api/scalability -> run all three across increasing input sizes for
                        the scalability / performance-comparison charts
```

```
"""
```

```
import sys
```

```
from pathlib import Path
```

```
BASE_DIR = Path(__file__).parent
```

```
sys.path.insert(0, str(BASE_DIR))
```

```
from flask import Flask, jsonify, render_template, request
```

```
from algorithms.dataset import generate_packages, save_csv, load_csv
```

```
from algorithms.knapsack import run_all
```

```
app = Flask(__name__)
```

```
DATA_PATH = BASE_DIR / "data" / "packages.csv"
```

```
# Generate a default dataset once at startup so /api/run has data to draw from
```

```

if not DATA_PATH.exists():
    save_csv(generate_packages(500, seed=42), DATA_PATH)

def get_dataset(n, seed=42):
    """Return n packages. Regenerates deterministically per (n, seed)."""
    return generate_packages(n, seed=seed)

def summarize(result):
    """Strip the full selected-items list down to a compact summary for JSON responses."""
    r = dict(result)
    selected = r.pop("selected", [])
    r["selected_sample"] = selected[:15] # cap payload size
    return r

@app.route("/")
def index():
    return render_template("index.html")

@app.route("/api/run", methods=["POST"])
def api_run():
    payload = request.get_json(force=True) or {}
    n_packages = int(payload.get("n_packages", 200))
    capacity = float(payload.get("capacity", 150))
    seed = int(payload.get("seed", 42))

    n_packages = max(5, min(n_packages, 3000))
    capacity = max(1.0, min(capacity, 5000.0))

    items = get_dataset(n_packages, seed=seed)
    results = run_all(items, capacity)

    return jsonify({
        "n_packages": n_packages,
        "capacity": capacity,
        "dp": summarize(results["dp"]),
        "greedy": summarize(results["greedy"]),
        "developed": summarize(results["developed"]),
    })

@app.route("/api/scalability")
def api_scalability():
    capacity = float(request.args.get("capacity", 150))
    max_n = int(request.args.get("max_n", 300))

```

```

step = int(request.args.get("step", 50))
seed = int(request.args.get("seed", 42))

max_n = max(step, min(max_n, 1500))
sizes = list(range(step, max_n + 1, step))

series = {"sizes": sizes, "dp": [], "greedy": [], "developed": []}

for n in sizes:
    items = get_dataset(n, seed=seed)
    results = run_all(items, capacity)
    for key in ("dp", "greedy", "developed"):
        r = results[key]
        series[key].append({
            "n": n,
            "time_ms": r["time_ms"],
            "memory_kb": r["memory_kb"],
            "total_value": r["total_value"],
            "capacity_utilization_pct": r["capacity_utilization_pct"],
            "optimality_gap_pct": r["optimality_gap_pct"],
            "num_selected": r["num_selected"],
        })

return jsonify(series)

if __name__ == "__main__":
    app.run(debug=True, host="0.0.0.0", port=5050)

```

OUTPUT IMAGES:

shivareddy1012/CSA0613-DAA

Run Zip In PowerShell

Logistics Allocation Console

Ask Gemini

127.0.0.1:5050

Which packages fit the truck?

A logistics resource-allocation problem, modeled as a 0/1 Knapsack: pick the subset of packages that maximizes delivery value without exceeding the vehicle's weight capacity. This console runs the exact Dynamic Programming baseline, a Greedy baseline, and our developed Adaptive Ratio-Filtered DP approach on the same instance, side by side.

01 - Run one instance

Choose a package-catalog size and vehicle capacity, then compare all three approaches.

Number of packages

Vehicle capacity (kg)

200

150 kg

Run comparison

Done.

Dynamic Programming

EXACT

	TOTAL VALUE	TIME	MEMORY
	5,600.76	1256.27 ms	357.9 KB
CAPACITY USED	100.0%	PACKAGES SELECTED	OPTIMALITY GAP
	41		0.00%

Greedy

HEURISTIC

	TOTAL VALUE	TIME	MEMORY
	5,596.65	0.41 ms	5.8 KB
CAPACITY USED	99.8%	PACKAGES SELECTED	OPTIMALITY GAP
	43		0.07%

Developed (ARF-DP)

DEVELOPED

	TOTAL VALUE	TIME	MEMORY
	5,600.76	402.39 ms	144.6 KB
CAPACITY USED	100.0%	PACKAGES SELECTED	OPTIMALITY GAP
	41		0.00%

Candidate pool reduced from 200 to 63 packages before running exact DP.

35°C Mostly cloudy

Search

ENG IN 30% 14:34 02-09-2026

shivareddy1012/CSA0613-DAA

Run Zip In PowerShell

Logistics Allocation Console

Ask Gemini

127.0.0.1:5050

02 - Scalability comparison

Runs all three approaches across increasing package-catalog sizes at a fixed vehicle capacity, to show how each scales.

Max packages

Vehicle capacity (kg)

400

150 kg

Run scalability test

Done.

Execution Time vs Input Size

Number of packages	Dynamic Programming (ms)	Greedy (ms)	Developed (ARF-DP) (ms)
50	~500	~100	~100
100	~1000	~100	~100
150	~1500	~100	~100
200	~2000	~100	~100
250	~2500	~100	~100
300	~3000	~100	~100
350	~3500	~100	~100
400	~4000	~100	~100

Peak Memory vs Input Size

Number of packages	Dynamic Programming (KB)	Greedy (KB)	Developed (ARF-DP) (KB)
50	~100	~10	~10
100	~200	~10	~10
150	~300	~10	~10
200	~400	~10	~10
250	~500	~10	~10
300	~600	~10	~10
350	~700	~10	~10
400	~800	~10	~10

Total Delivery Value vs Input Size

Number of packages	Dynamic Programming (Value)	Greedy (Value)	Developed (ARF-DP) (Value)
50	~4000	~3500	~4000
100	~4500	~3800	~4500
150	~5000	~4100	~5000
200	~5500	~4400	~5500
250	~6000	~4700	~6000
300	~6500	~5000	~6500
350	~7000	~5300	~7000
400	~7500	~5600	~7500

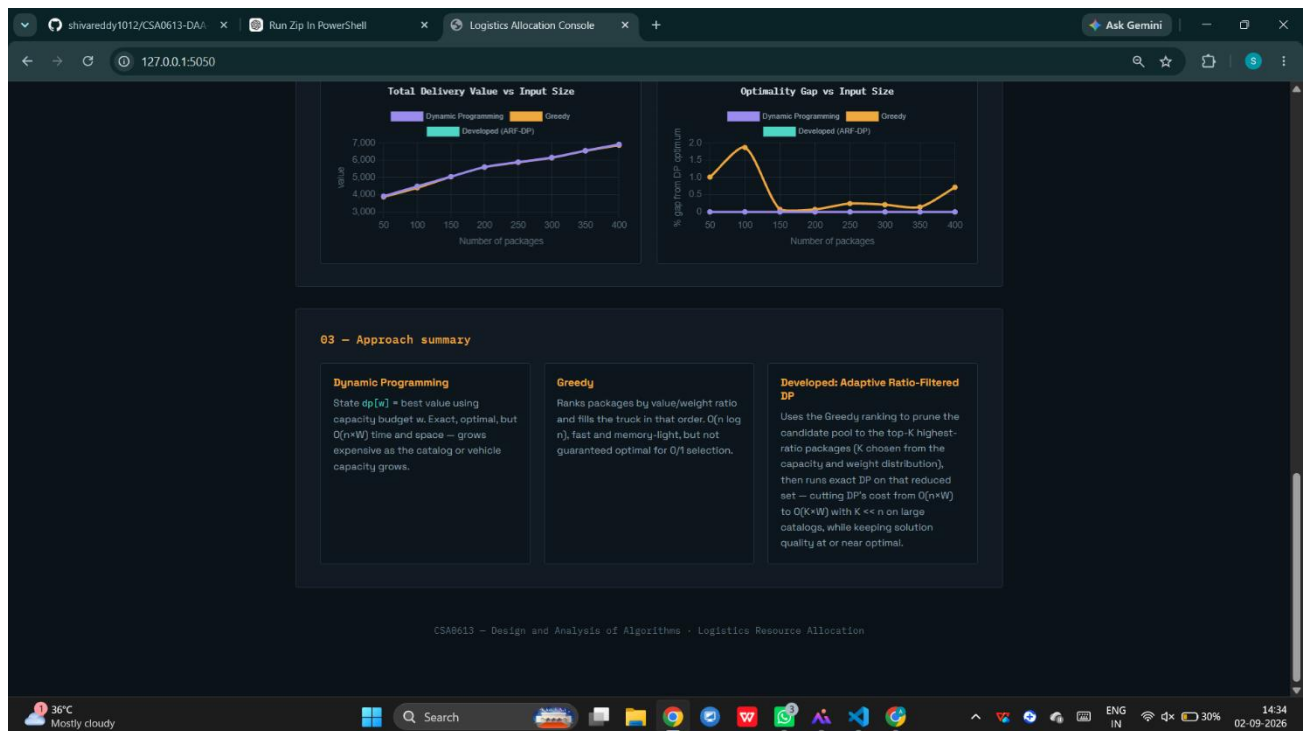
Optimality Gap vs Input Size

Number of packages	Dynamic Programming (%)	Greedy (%)	Developed (ARF-DP) (%)
50	0.00	~1.00	0.00
100	0.00	~1.80	0.00
150	0.00	~0.50	0.00
200	0.00	~0.20	0.00
250	0.00	~0.10	0.00
300	0.00	~0.05	0.00
350	0.00	~0.02	0.00
400	0.00	~0.01	0.00

35°C Mostly cloudy

Search

ENG IN 30% 14:34 02-09-2026



F. Assessment Rubric

Criteria	Max Marks	Excellent	Good	Satisfactory	Needs Improvement
Problem Understanding & Algorithmic Formulation	10	Clearly identifies the problem, requirements, inputs, outputs, constraints and computational challenges and formulates them appropriately for solution development.	Problem and major requirements are correctly formulated with minor gaps.	Basic formulation is provided, but requirements or constraints are partially identified.	Problem is poorly understood or inadequately formulated.
Algorithmic Solution Design	15	Designs a clear, logical and feasible solution strategy with appropriate algorithmic techniques, pseudocode/flowchart and well-justified design decisions.	Appropriate solution design with minor gaps in logic, representation or justification.	Basic solution design is presented but lacks completeness or clear justification.	Solution design is inappropriate, incomplete or unclear.
Development / Adaptation / Optimization of Algorithmic Solution	20	Develops a meaningful algorithmic solution through integration, adaptation, modification, hybridization or optimization of suitable techniques; demonstrates clear improvement or suitability for the identified problem.	Develops an appropriate solution with meaningful adaptation or improvement, with minor limitations.	Solution demonstrates limited adaptation or development beyond standard implementation.	Merely reproduces an existing algorithm with little or no meaningful development or adaptation.
Implementation & Modern Tool Usage	15	Developed solution is correctly and efficiently implemented using appropriate programming/computational tools and handles relevant input conditions reliably.	Implementation is functional with minor logical or efficiency issues.	Core implementation works but contains limitations or incomplete functionality.	Implementation is incomplete, unreliable or non-functional.

Efficiency, Testing & Validation	15	Performs appropriate complexity evaluation and systematic testing using suitable data/input sizes; validates correctness, efficiency and scalability with clear evidence.	Complexity evaluation and testing are mostly appropriate with minor gaps.	Basic testing and efficiency evaluation are provided but lack sufficient validation.	Testing, complexity evaluation or validation is inadequate or substantially incorrect.
Performance Improvement, Comparison & Final Decision	15	Demonstrates the effectiveness of the developed solution through appropriate comparison; critically evaluates performance,	Good performance comparison and justification	Basic comparison is provided, but improvement or final	Improvement is not demonstrated or the final algorithmic
		limitations and trade-offs and justifies the final algorithmic decision with evidence.	with minor limitations.	justification lacks depth.	decision is unsupported.
Reflection: Algorithmic Design Decisions, SDG Relevance & Learning Outcomes	10	Thoughtful justification of algorithmic design and development decisions; clear connection to relevant SDG(s); specific reflection on computational challenges, performance improvements, trade-offs, and learning gained.	Appropriate justification of algorithmic decisions; SDG relevance, challenges, improvements, and learning are discussed but lack depth.	Reflection is present but generic; limited justification of algorithmic decisions, improvements, SDG relevance, or learning outcomes.	No genuine reflection is provided, or the reflection lacks meaningful connection to algorithm development, SDG relevance, challenges, or learning outcomes.
Total	100				

G. CO–PO–Assessment Mapping

Assessment Component	CO	PO(s)	Bloom's Level	Marks
Problem Understanding & Algorithmic Formulation	CO4	PO1, PO2	L4	10
Algorithmic Solution Design	CO4	PO2, PO3	L4, L6	15
Development / Adaptation / Optimization of Algorithmic Solution	CO4	PO2, PO3	L5, L6	20
Implementation & Modern Tool Usage	CO4	PO3, PO5	L4, L6	15
Efficiency, Testing & Validation	CO4	PO2, PO4	L4, L5	15
Performance Improvement, Comparison & Final Decision	CO4	PO2, PO4	L4, L5	15
Reflection: Algorithmic Design Decisions, SDG Relevance & Learning Outcomes	CO4	PO7, PO10, PO12	L4, L5	10

H. Faculty Evaluation & Continuous Improvement CO Attainment

Performance Level	Criterion
Level 3	$\geq 70\%$
Level 2	60–69%
Level 1	50–59%
Level 0	$< 50\%$

Target CO Attainment: _____

Actual CO Attainment: _____

Faculty Analysis

Areas in which students performed well:

Areas requiring improvement:

Corrective / Improvement Action: